

qTest-JIRA Workflows – Ultimate Integration Guide



Page is about?

*This comprehensive document outlines the step-by-step, universal guide for integrating **qTest with Jira**, managing **QA workflows**, and establishing **full traceability** between stories, tests, and defects.*

1. Configuring the Integration

Integration allows seamless bi-directional syncing of:

- Jira requirements ↔ qTest test cases
- Jira defects ↔ qTest test runs

Key Benefits:

- Sync visibility across both tools
- qTest test pass/fail shown in Jira
- Test cases linked directly to requirements
- Defects auto-linked to test runs and requirements

2: Configuring Jira for qTest Integration

Before integrating with qTest, your **Jira project** must meet the following configuration criteria:

Required Jira Project Setup

1. Work Collection Defined

- You must use either:
 - **Sprint** (Scrum teams)
 - **Fix Version** (Kanban or Release-based teams)
- These are used by qTest to define “Releases.”

2. Requirement Ticket Type Identified

- Preferred: Story
- Can be customized to any type
- These sync to qTest as **Requirements**

3. Defect/Bug Ticket Type Identified

- Preferred: Bug or Defect
- Can include other types (e.g., Escaped Defect)
- These sync to qTest as **Defects** linked to test runs

4. Generate Jira API Token

Needed for establishing secure integration between qTest and Jira

Steps to create API token:

- Go to: [Atlassian Security Settings](#)
- Click **Create and manage API tokens**
- Choose a memorable label (e.g., qTest Integration)
- Click **Create**, then **Copy**
- Store in **PasswordState** (cannot copy again later)

5. Cross-Team Project Considerations

- If multiple Jira projects are involved:
 - Confirm which project uses **Sprints** vs **Fix Versions**
 - Standardize requirement and defect issue types (recommended)
 - The Jira account used for integration must have **access to all linked projects**
 - Only **one API token** per qTest project is supported

3: Configuring qTest for Jira Integration

Once your Jira project is prepared, follow these steps to complete integration setup within qTest:

A. Connect Jira to qTest

- a. Navigate to:
qTest Manager Administration External Systems Jira
- b. Click: **Add Jira Connection**
- c. Fill in:
 - **Connection Name:** e.g., UP-Services-Jira
 - **Server URL:** <https://ascensuscollegesavings.atlassian.net>
 - **Web URL:** same as above
 - **Username:** Jira account email used to generate the token
 - **Token:** Paste your API token
- d. Click **Test Connection**, then **Save & Configure**

B. Configure Defect Integration

- a. Navigate to: **DEFECT INTEGRATION** tab
- b. Click **Add Defect Type**
- c. Select the **Jira project**
- d. Choose the defect issue type (e.g., Bug)
- e. Check **"Also populate Jira defects not linked to Test Runs"**
- f. Repeat for any other issue types you use for defects

Repeat this step per Jira project if using multiple

C. Configure Requirement Integration

- a. Navigate to: **REQUIREMENT INTEGRATION** tab
- b. Toggle **Activate requirement integration**
- c. Click **Add Requirement Type**
- d. Select the **Jira project**
- e. Choose issue type (e.g., Story)
- f. Click **OK**

Repeat per Jira project if needed

D. Configure Release Integration

- a. Navigate to: **RELEASE INTEGRATION** tab
- b. Toggle **Activate release integration**
- c. Click **Add Release Type**
- d. Select the Jira project
- e. Choose:
 - **Sprint** (for Scrum)
 - **Fix Version** (for Kanban or other teams)
- f. Click **OK**

Repeat per Jira project as applicable

E. Finalize & Sync

- Click **Save & Activate**
- If prompted, **Refresh Browser**
- Confirm **Synchronization Status: OK**
- Optionally: click **Fetch/Retrieve** to force data sync

F. Add Custom Fields to Test Cases

In qTest, navigate to **Field Settings > Test Case Object Type** and:

- a. Add:
 - **Automation Status** (Combo Box)
 - Values: Unknown (default), Needs Automation, Automated, Will Not Automate
 - **Related Test Automation** (Text)
- b. Check **Searchable** for both

4. Scrum vs Kanban Considerations

Aspect	Scrum Workflow	Kanban Workflow
Test Scope	Sprint-level planning & suites	Ticket/Epic-level, bucketed as release
Test Suites	Created per sprint	Created per feature/ticket
Regression	Done per sprint or release	Done per ticket or regular batch

For Kanban: use a **"Work in Progress"** release bucket to test tickets not yet released.

5. Primary Workflow Scenarios for qTest-JIRA Integration

This section outlines **three core end-to-end scenarios** for how agile teams should work with qTest and JIRA together to maintain traceability, ensure test coverage, and manage defects across the lifecycle of software development. This includes:

- New Feature Development
- Updating Existing Features
- Handling Unrelated Defects

A. New Feature Development

Applicable Roles: *Business Analyst, Product Owner, Architect, Quality Engineer*

Step-by-Step Workflow

1. **Capture Requirements in JIRA**
 - Create **Story** type tickets with detailed acceptance criteria.
 - Ensure stories are aligned with planned work and product expectations.
2. **Sync Requirements to qTest**
 - Requirements auto-sync to **qTest Requirements** module.
 - Organize into **folders/modules** by feature or domain.
3. **Add Requirements to Sprint/Release**
 - Assign JIRA stories to active sprint or fix versions.
 - These show up as **Releases** in qTest's **Test Plan/Test Execution** tabs.
4. **Create/Link Test Cases in qTest**
 - QA/BA create new test cases OR associate existing ones to each requirement.
 - Use + Add in **Linked Test Cases** section.
5. **Review & Approve Test Cases**
 - Business Analyst and QA validate all test steps and completeness.
 - Mark test cases as *Approved* in qTest for execution.
6. **Assemble a Test Suite**
 - In **Test Execution**, create a new suite under the correct sprint/release.
 - Add all relevant test cases, mark as smoke/regression if needed.
7. **Run Test Suite**
 - Launch **Test Pad**, execute tests, and mark pass/fail.
 - Attach defects directly from failing steps (auto-linked in JIRA & qTest).
8. **Report & Iterate**
 - Use qTest dashboards for summary.
 - Communicate test coverage, blockers, and pass rate to team.
 - Iterate tests if requirements or implementations change.
9. **Add Test Automation Tags (if applicable)**
 - Update Automation Status Automated/Needs Automation.
 - Add automation path/link in Related Test Automation.

B. Updating or Modifying Existing Features

Applicable Roles: *Product Owner, Scrum Master, QA, Business Analyst*

Step-by-Step Workflow

1. **Identify Changed Requirements**
 - If prior stories lack traceability, **create new JIRA Story** with version suffix (e.g., -v2).
 - Link to the original story in JIRA.

2. Repeat Standard Workflow Steps

- Reuse:
 - Capture Requirements JIRA
 - Sync to qTest
 - Create/Update Test Cases
 - Test Case Review
 - Assemble New/Updated Test Suite

3. Review Existing Test Cases

- Reassess test cases already linked to the previous version.
- Update where necessary and re-approve.

4. Add Regression Tests

- Include test cases targeting unchanged functionality or fixed defects.
- Migrate or clone older test runs to the new suite.

5. Notify Other Teams

- Share updated test suites if their area depends on this change.
- Include links in your release notes or comments on related epics.

C. Defects Found During Unrelated Work

Applicable Roles: *Any Agile Team + QA*

Step-by-Step Workflow

1. Log Defect in Correct JIRA Project

- Do NOT raise defects in qTest directly.
- Use the **most likely owner team's** JIRA board.

2. Determine Ownership

- Try to trace the origin using:
 - Feature/module ownership history.
 - Test case history in qTest.
 - Past release data.

3. Tag Product/Dev Owner

- Notify respective **Scrum Master** and **PO** with JIRA link and traceability context.

4. Backfill Missing Traceability

- If the defect lacks related test cases:
 - Create new test case in qTest manually.
 - Link to requirement/story if applicable.
 - Tag automation needs if repetitive in nature.

5. Update Test Suites

- Add new test cases to regression or impacted feature test suites.
- Mark them for re-execution post-fix.

6. Document Learnings

- Capture root cause and missed coverage in JIRA.
- Optionally, link in Confluence for future retrospective review.

Success Criteria for All Workflows

Action	Success Indicator
Requirements captured	All planned stories have detailed, synced qTest requirements
Test cases mapped	Every requirement links to 1 approved test case
Traceability established	Defects Test Runs Test Cases Requirements
Automation coverage	Test cases tagged with automation status, and automated if possible
QA feedback loop	Issues logged, iterations performed, communication maintained
Test suite auditability	Test suite versions and execution history logged & accessible

6. Best Practices

- Organize test cases & requirements using **Modules**

- Every requirement 1+ test case(s) (manual or automated)
- Get test cases reviewed & **Approved** by QA/BA
- Flag automation candidates & update Automation Status
- Create test runs using **Test Execution** view capture execution context

Escaped/Unrelated Defect Handling

If a defect isn't tied to a change by your team:

- Create a Jira ticket (not from qTest)
- Link defect only to requirement & test after triage
- Alert the **owning team** or escalate for ownership decision